

PRODUCTION INCIDENT CASEBOOK

20 REAL INCIDENTS. **20** FAILURE CHAINS. **20** ENGINEERING LESSONS.

- ✓ REAL-WORLD OUTAGES
- ✓ ROOT CAUSE ANALYSIS
- ✓ TIMELINES AND IMPACT
- ✓ INVESTIGATION STEPS
- ✓ MITIGATION AND RECOVERY
- ✓ ENGINEERING LESSONS
- ✓ PREVENTION STRATEGIES



CLOUD



INFRASTRUCTURE



KUBERNETES



DATABASES



NETWORKING



SYSTEMS



OBSERVABILITY



SECURITY

REAL ENGINEERING PROBLEMS.
REAL THINKING. REAL SOLUTIONS.

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW

AWS S3 US-EAST-1, 2017

OPERATIONAL COMMAND REMOVES FAR MORE CAPACITY THAN INTENDED

1. INCIDENT OVERVIEW

On February 28, 2017, a routine operational task in the AWS S3 US-EAST-1 region accidentally removed far more servers than intended. This triggered widespread errors and service unavailability for multiple AWS services that depend on S3.

3. IMPACT

- S3 experienced high error rates and unavailability
- Multiple AWS services were affected (e.g., EC2, RDS, Lambda, Elastic Beanstalk)
- Many customer applications could not read or write data
- Some services experienced cascading failures
- Impact lasted for several hours

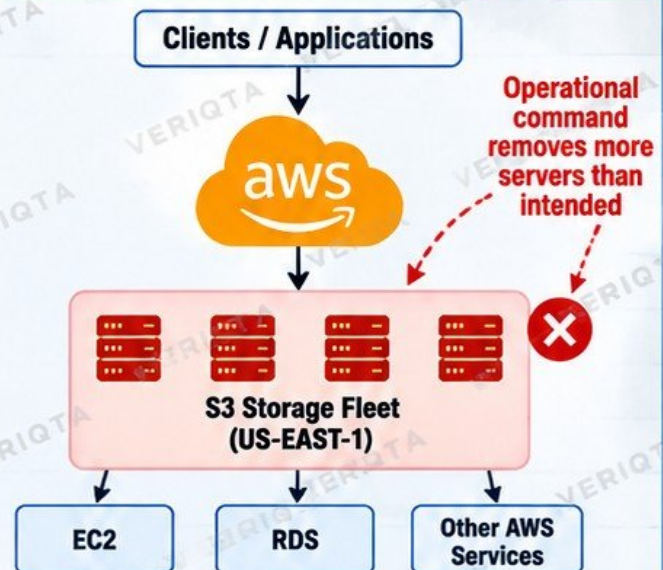
4. TIMELINE (UTC)

14:49	Initial elevated error rate detected
15:27	Significant increase in error rates
15:55	AWS confirmed the issue (S3)
17:22	Error rates began to improve
20:39	S3 returned to normal operations

7. KEY ENGINEERING LESSONS

- ✓ Always validate inputs for operational commands
- ✓ Use additional safeguards for destructive operations
- ✓ Improve blast radius controls
- ✓ Design systems to handle large-scale failures
- ✓ Have clear rollback and recovery procedures
- ✓ Test operational procedures regularly
- ✓ Understand and map service dependencies

2. SIMPLIFIED ARCHITECTURE



5. ROOT CAUSE

An authorized AWS engineer, following an established operational playbook, entered an incorrect command input. This unintentionally removed a large number of S3 servers instead of the smaller set that was intended.

6. WHAT MADE IT WORSE

- The command removed significantly more capacity
- Redundancy and automated recovery could not fully mitigate the large-scale removal
- S3 is a highly critical service with many dependencies
- Recovery required substantial time to restore capacity

8. PRODUCTION TAKEAWAYS

- ✓ Human error can happen, even with experienced engineers
- ✓ A single operational mistake can have a huge blast radius
- ✓ Build defense in depth for critical services
- ✓ Monitor, detect and communicate quickly
- ✓ Design for failure and plan for recovery
- ✓ Regularly review operational runbooks and permissions

FAILURES TEACH.
PREPARATION PREVENTS.

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

GITLAB DATABASE DISASTER, 2017

POSTGRESQL REPLICATION, ACCIDENTAL DELETION, FAILED BACKUPS, DATA LOSS

1. INCIDENT OVERVIEW

On January 31, 2017, GitLab experienced a major database outage. An accidental deletion on the primary PostgreSQL database, combined with replication issues and failed backups, led to permanent data loss for projects, issues, merge requests and other data.

3. IMPACT

-  Significant data loss (projects, issues, merge requests, CI data, etc.)
-  Many users could not access their data
-  GitLab.com was unavailable for several hours
-  Some data could not be recovered

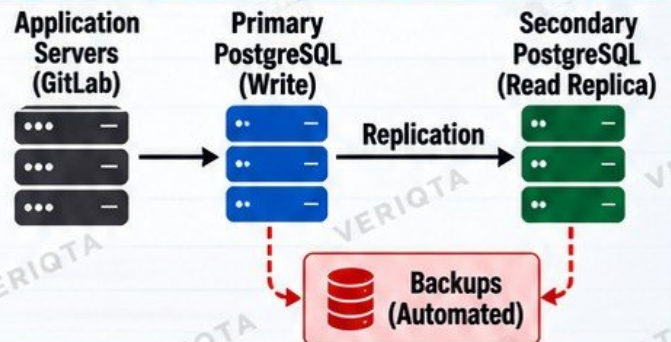
5. FAILURE CHAIN

- 1 Accidental deletion on primary database
- ↓
- 2 Replication did not have the latest data
- ↓
- 3 Backups were incomplete and could not restore the deleted data
- ↓
- 4 Resulted in permanent data loss

7. WHAT MADE IT WORSE

- Lack of immediate detection
- Backups were not regularly tested for restore
- No safe-guards like read-only mode or extra confirmation
- Multiple systems failed (primary, replica, backups)
- Recovery time was long and complex

2. SIMPLIFIED ARCHITECTURE (BEFORE INCIDENT)



4. WHAT HAPPENED (TIMELINE UTC)

- 13:49 | Routine maintenance on database server
- 14:01 | Accidental command issued, deleting important data
- 14:10 | Replication issues prevented the secondary from having up-to-date data
- 14:20 | Backup process also failed
- 15:27 | Incident declared and recovery efforts started
- Later | Some data permanently lost

6. ROOT CAUSES

- Accidental execution of a destructive command
- Replication lag and configuration gaps
- Backup process failure
- Insufficient safeguards for destructive operations
- Inadequate verification before maintenance tasks

8. ENGINEERING LESSONS

- ✓ Always validate commands, especially destructive ones
- ✓ Ensure replicas are up-to-date and monitored
- ✓ Regularly test backups and verify restore procedures
- ✓ Implement additional safety checks for database operations
- ✓ Monitor replication lag and backup health
- ✓ Have clear incident response and communication plan

**REAL INCIDENTS BUILD
BETTER ENGINEERS.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

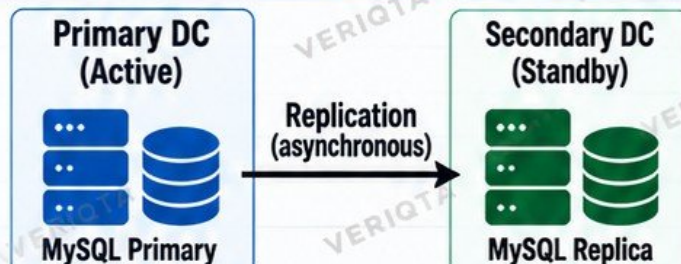
GITHUB NETWORK PARTITION, 2018

MYSQL FAILOVER, SPLIT TOPOLOGY, REPLICATION AND CONSISTENCY

1. INCIDENT OVERVIEW

On October 21, 2018, GitHub experienced a major outage caused by a network partition between its data centers. The partition isolated parts of the database infrastructure, leading to a complex MySQL failover and data consistency challenges. GitHub prioritized data integrity over speed of recovery.

2. SIMPLIFIED ARCHITECTURE (BEFORE INCIDENT)



3. IMPACT

-  GitHub.com was unavailable for several hours
-  Users could not access repositories, create pull requests or perform most actions
-  Risk of data inconsistency between data centers
-  Engineers had to perform complex recovery steps
-  Maintained data integrity, but at the cost of a longer outage

4. WHAT HAPPENED (TIMELINE UTC)

- 13:52** | Network partition detected between data centers
- 14:03** | Primary database became isolated
- 14:04** | Failover initiated to promote replica
- 14:24** | Complications with replication and inconsistent data
- 15:03** | Engineers analyzed data divergence
- 17:24** | Began restoring services with consistent data
- 21:52** | GitHub.com fully operational

* Times are approximate based on GitHub's post-incident analysis.

5. FAILURE CHAIN

- 1 Network partition isolated primary database
- 2 Failover promoted a replica to primary
- 3 Writes occurred in isolated environments
- 4 Data divergence between primary and replica
- 5 Complex recovery to ensure consistency

6. ROOT CAUSES

- Network partition between data centers
- Asynchronous replication allowed divergence
- Complex topology made recovery slow
- Lack of automated, safe failover for this scenario
- Protecting data integrity required manual intervention

7. WHAT MADE IT WORSE

- Split-brain risk with multiple primaries
- Data consistency could not be guaranteed quickly
- Recovery required careful analysis and manual steps
- Engineers had to disable some services during recovery
- Large and complex database environment

8. ENGINEERING LESSONS

- ✓ Design for network partitions, they will happen
- ✓ Understand your replication model and its limitations
- ✓ Automate safe failover and split-brain protection
- ✓ Build runbooks for complex recovery scenarios
- ✓ Monitor replication health and data consistency
- ✓ Sometimes the right decision is to be slow and safe

**DATA INTEGRITY
BUILDS TRUST.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

CLOUDFLARE WAF GLOBAL OUTAGE, 2019

CATASTROPHIC REGEX BACKTRACKING AND FLEET-WIDE CPU EXHAUSTION

1. INCIDENT OVERVIEW

On July 2, 2019, Cloudflare experienced a global outage caused by a new Web Application Firewall (WAF) rule containing a catastrophic regular expression (regex). The regex triggered excessive CPU usage on every request, leading to high CPU across the global fleet and widespread service disruption for multiple customers.

3. IMPACT



Many customer websites and services were unavailable or returning errors



Affected customers included major companies across different industries



Lasted for several hours

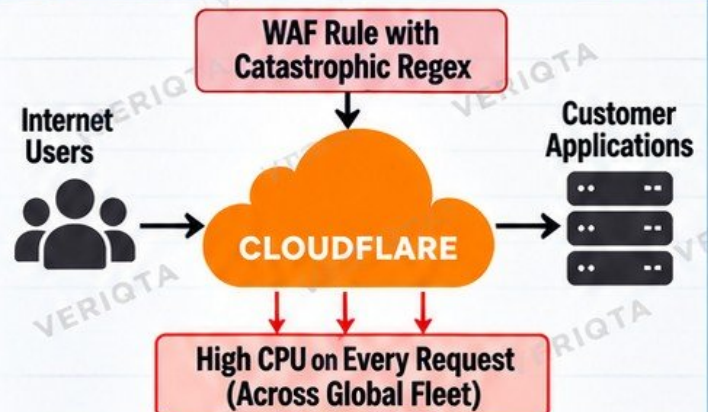


Significant global impact due to Cloudflare's wide footprint



Demonstrated how a single configuration change can affect an entire global system

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

12:42	New WAF rule deployed to production
12:44	CPU usage started increasing across the fleet
12:50	Widespread errors reported by customers
13:17	Issue identified as a regex causing high CPU
13:27	WAF rule disabled
14:42	Services recovered

* Times are approximate based on Cloudflare's post-incident analysis.

5. ROOT CAUSE

- A new WAF rule contained a catastrophic regex that caused excessive backtracking
- The regex was syntactically valid and passed tests, but had pathological performance
- Every matching request consumed high CPU
- The rule was quickly deployed globally because WAF rules follow a different deployment path

6. WHAT MADE IT WORSE

- WAF rules are deployed globally and immediately
- Normal staged deployment (canary) was bypassed
- Tests did not detect the high CPU behavior
- High CPU affected all edge servers simultaneously
- Rapid traffic growth amplified the impact

7. ENGINEERING LESSONS

- ✓ Test for performance, not just correctness
- ✓ Have a safe deployment strategy, even for security rules
- ✓ Use tooling to detect pathological regex patterns
- ✓ Implement blast radius controls for global changes
- ✓ Consider automated rollback based on CPU or error rates
- ✓ Understand the unique risks of security rule deployments

8. KEY TAKEAWAYS

- ✓ A small configuration change can have a global impact
- ✓ Regular expressions can be dangerous
- ✓ Performance testing is critical for production changes
- ✓ Different change types may need different safeguards
- ✓ Monitor system metrics closely after deployments
- ✓ Prepare for fast rollback mechanisms

**RESILIENT SYSTEMS
LEARN FROM REAL INCIDENTS.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

CLOUDFLARE AND THE VERIZON ROUTE LEAK, 2019

BGP ROUTE LEAK AND INTERNET ROUTING FAILURE

1. INCIDENT OVERVIEW

On June 24, 2019, a BGP route leak from Verizon unintentionally advertised Cloudflare's more specific routes to the wrong peers. This caused a large portion of global internet traffic to be misrouted through Verizon's network, disrupting access to many websites and services, including some of Cloudflare's customers.

3. IMPACT



Many websites and online services became slow or unreachable



Affected multiple industries and countries



Lasted for about 1 hour



Large volume of global internet traffic was misrouted



Highlighted the risk of BGP route leaks to the entire internet ecosystem

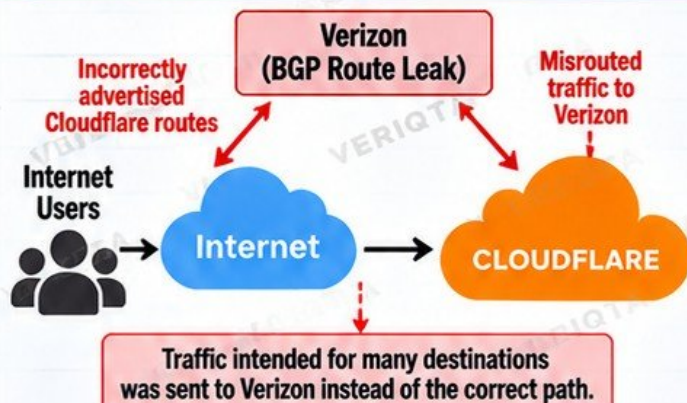
5. ROOT CAUSES

- Verizon accidentally advertised Cloudflare's more specific BGP routes to the wrong peers
- Lack of proper route filtering and controls
- Human and process error in BGP configuration
- BGP's trust model allowed the incorrect routes to propagate quickly across the internet

7. ENGINEERING LESSONS

- ✓ Implement strict BGP route filtering and validation
- ✓ Monitor for route leaks and abnormal route announcements
- ✓ Use prefix length limits and route origin validation (RPKI)
- ✓ Have incident response plans for routing issues
- ✓ Collaborate with upstream providers and peers
- ✓ Understand how quickly BGP changes can affect the world

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

- | | |
|--|--|
| 21:24
21:28
21:32
22:05
22:24 | Verizon began advertising Cloudflare's routes to the wrong peers
 Traffic started to be misrouted globally
 Widespread service disruption reported
 Verizon stopped the incorrect route advertisement
 Internet routing returned to normal |
|--|--|

* Times are approximate based on public reports from Cloudflare.

6. WHAT MADE IT WORSE

- BGP automatically trusts route announcements
- The leaked routes were more specific, so they were preferred by other networks
- The routes propagated globally within minutes
- There is no centralized control over the global internet
- Many networks accepted and propagated the incorrect routes

8. KEY TAKEAWAYS

- ✓ A single routing mistake can impact a large part of the internet
- ✓ BGP has no built-in security, so filtering and monitoring are critical
- ✓ More specific routes are preferred, even if they are incorrect
- ✓ Route leaks can happen to any network, including major providers
- ✓ Global internet stability depends on cooperation and good practices
- ✓ Always monitor your network and have a plan to react quickly

**ROUTING MISTAKES
CAN BREAK THE INTERNET.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

FASTLY GLOBAL CDN OUTAGE, 2021

LATENT SOFTWARE DEFECT TRIGGERED BY VALID CUSTOMER CONFIGURATION

1. INCIDENT OVERVIEW

On June 8, 2021, Fastly experienced a global outage after a latent software defect in its configuration processing was triggered by a valid customer configuration. This caused a large portion of Fastly's edge network to return errors, impacting many major websites and online services worldwide.

3. IMPACT



Many popular websites and services were unavailable or returned errors



Affected customers included major companies across multiple industries (e.g., Amazon, Reddit, The New York Times, Spotify, etc.)



Lasted for approximately 49 minutes



Around 85% of Fastly's network returned errors at the peak



Significant global impact due to Fastly's role as a major CDN provider

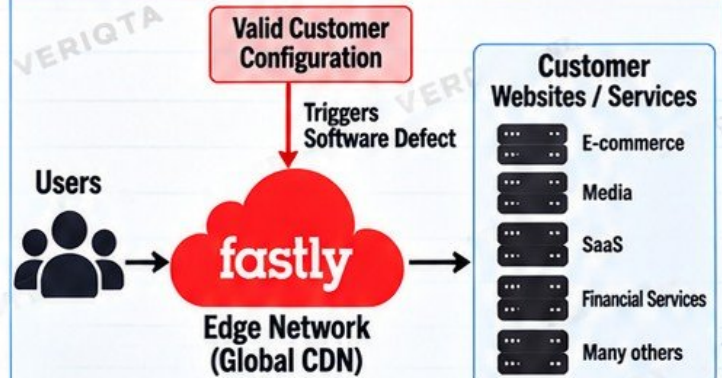
5. ROOT CAUSE

- A latent software defect in Fastly's configuration processing logic
- Triggered by a valid customer configuration (using a specific VCL (Varnish Configuration Language) syntax)
- The configuration caused unbounded resource consumption on edge servers
- The defect had been in the codebase for months
- It was not detected by existing tests, code review or monitoring before deployment

7. ENGINEERING LESSONS

- ✓ Test for unexpected inputs, not just invalid ones
- ✓ Improve test coverage for configuration parsing
- ✓ Use staged and progressive rollouts with blast radius controls
- ✓ Implement better safeguards for edge configurations
- ✓ Enhance monitoring to detect abnormal resource usage early
- ✓ Understand the impact of customer-driven configurations

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

- 16:42 | Issue began after a customer configuration change
- 16:44 | Errors started appearing across the network
- 16:47 | Significant increase in error rates
- 16:49 | ~85% of Fastly's network returning errors
- 17:16 | Engineers identified the root cause
- 17:27 | Fix implemented
- 17:31 | Services recovered

* Times are approximate based on Fastly's post-incident report.

6. WHAT MADE IT WORSE

- The configuration was valid, so it passed initial checks
- The defect was not caught in testing or staging
- The change was deployed globally and quickly
- Monitoring did not detect the issue early enough
- The defect affected a large portion of the fleet simultaneously
- High traffic amplified the impact

8. KEY TAKEAWAYS

- ✓ Valid customer input can still trigger critical failures
- ✓ Latent defects can remain undetected for a long time
- ✓ Global systems require strong blast radius controls
- ✓ Testing and monitoring must cover real-world scenarios
- ✓ Fast detection and rollback are essential
- ✓ Assume that any change can have a large global impact

**SMALL CHANGES
CAN HAVE A GLOBAL IMPACT.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Meta **META GLOBAL OUTAGE, 2021**

BACKBONE ISOLATION, BGP WITHDRAWAL, DNS FAILURE AND INACCESSIBLE RECOVERY SYSTEMS

1. INCIDENT OVERVIEW

On October 4, 2021, Meta (Facebook, Instagram, WhatsApp and other services) experienced a major global outage after a network configuration change caused the company's backbone to become isolated. This led to BGP route withdrawal, DNS failure, and made many internal tools and recovery systems inaccessible, significantly delaying restoration of services.

3. IMPACT

-  Facebook, Instagram, WhatsApp and other Meta services were unavailable globally
-  Billions of users could not access services
-  Lasted for approximately 5 to 6 hours
-  Significant global impact on communication, businesses and online services
-  Internal engineers also lost access to many critical tools, making recovery more difficult

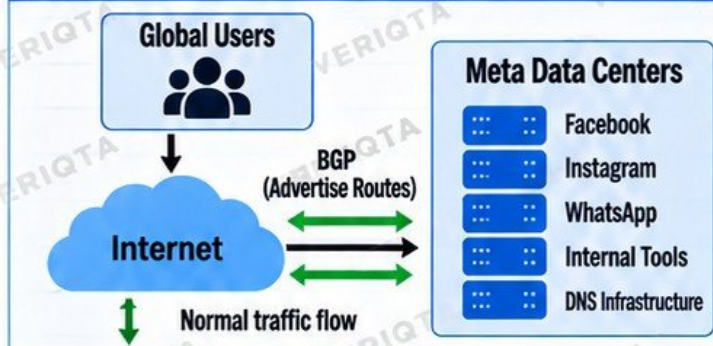
5. ROOT CAUSES

- Faulty network configuration change
- Removal of critical BGP routes
- Backbone isolation across data centers
- DNS infrastructure became unreachable
- Lack of safeguards to prevent total loss of connectivity
- Recovery systems depended on the same network

7. ENGINEERING LESSONS

- ✓ Implement safeguards for network configuration changes
- ✓ Use staged rollouts and validation for critical network updates
- ✓ Ensure out-of-band access to critical systems
- ✓ Design recovery systems that are independent
- ✓ Monitor BGP announcement and route changes
- ✓ Plan for total network isolation scenarios

2. SIMPLIFIED ARCHITECTURE (BEFORE INCIDENT)



4. WHAT HAPPENED (TIMELINE UTC)

- 15:39** | Network configuration change deployed
- 15:42** | Backbone became isolated
- 15:44** | BGP routes were withdrawn
- 15:45** | DNS stopped resolving for Meta domains
- 16:00** | Internal tools became inaccessible
- 19:00** | Engineers began restoring connectivity
- 21:49** | Services gradually came back online

* Times are approximate based on publicly available reports.

6. WHAT MADE IT WORSE

- No automatic protection against full route withdrawal
- Engineers lost access to internal tools and communication
- DNS failure made both external and internal access difficult
- Multiple dependent systems failed simultaneously
- Recovery required manual and out-of-band access
- The scale of the outage affected all major services at once

8. KEY TAKEAWAYS

- ✓ Small configuration changes can have massive impact
- ✓ Network isolation can quickly cascade to DNS and services
- ✓ Always have out-of-band access for recovery
- ✓ Design systems to fail independently
- ✓ Test disaster recovery for extreme scenarios
- ✓ Assume that you could lose internal access during an incident

**RESILIENCE COMES
FROM PREPARATION.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



CLOUDFLARE BGP OUTAGE, 2022

NETWORK POLICY ORDERING WITHDRAWS CRITICAL PREFIXES

1. INCIDENT OVERVIEW

On June 21, 2022, Cloudflare experienced a major outage after a change to its network configuration caused critical IP prefixes to be withdrawn from the global internet. A network policy ordering issue led to routes being announced incorrectly, making many Cloudflare data centers unreachable and causing a large increase in errors for customers worldwide.

3. IMPACT



Many websites and services using Cloudflare were unavailable or returning errors



Affected customers included major companies across multiple industries



Lasted for approximately 1 hour and 3 minutes

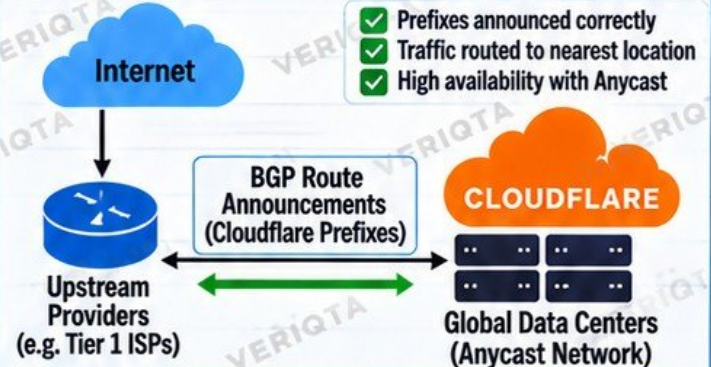


Error rates increased significantly across the global network



19 data centers went offline (about 4% of locations but ~50% of traffic)

2. SIMPLIFIED ARCHITECTURE (BEFORE INCIDENT)



4. TIMELINE (UTC)

16:42	Configuration change deployed
16:44	Critical prefixes began to be withdrawn
16:47	Many data centers became unreachable
16:50	Customer errors increased significantly
17:21	Engineers identified the issue
17:37	Fix implemented (policy order corrected)
17:45	Traffic gradually restored

* Times are approximate based on Cloudflare's post-incident report.

5. ROOT CAUSES

- Network policy ordering issue in route configuration
- Critical prefixes were unintentionally withdrawn
- Change passed internal checks but did not catch the policy ordering problem
- Insufficient safeguards for this specific type of change
- Complexity of global network configuration

6. WHAT MADE IT WORSE

- The change affected many locations simultaneously
- Automated checks did not detect the issue
- High dependency on correct BGP announcements
- Large portion of traffic was concentrated on the affected prefixes
- Recovery depended on reverting the configuration and re-advertising routes

7. ENGINEERING LESSONS

- ✓ Add more granular validation for network policy changes
- ✓ Test BGP changes in realistic environments
- ✓ Implement stronger safeguards for critical prefixes
- ✓ Monitor route announcements continuously
- ✓ Have a clear and fast rollback plan for network changes
- ✓ Understand the blast radius of network configuration updates

8. KEY TAKEAWAYS

- ✓ Small configuration changes can have a massive global impact
- ✓ Network policy ordering matters
- ✓ Validation and testing must cover real-world scenarios
- ✓ Monitor external visibility (BGP announcements)
- ✓ Design defense in depth for network changes
- ✓ Be prepared to recover quickly from routing issues

**ROUTES CONNECT
THE WORLD. HANDLE
CHANGES WITH CARE.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta

veriqta

veriqta

veriqta

veriqta

veriqta



CLOUDFLARE DATA CENTER POWER FAILURE, 2023

POWER, DISASTER RECOVERY AND HIDDEN CONTROL-PLANE DEPENDENCIES

1. INCIDENT OVERVIEW

On November 14, 2023, Cloudflare experienced a significant outage after a power failure at one of its data centers. The loss of power impacted critical systems and exposed hidden dependencies in the control plane, leading to service disruptions across multiple products and regions. The incident highlighted the complexity of disaster recovery and the importance of removing single points of failure in the control plane.

3. IMPACT

- Multiple Cloudflare services were degraded or unavailable
- Affected customers included businesses, developers and major online platforms
- Lasted for several hours
- Impacted global traffic and edge services
- Highlighted the risk of control-plane dependencies and data center level failures

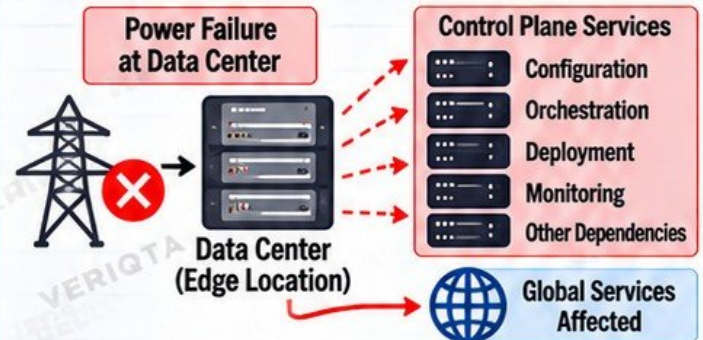
5. ROOT CAUSES

- Power failure at a data center
- Hidden control-plane dependencies
- Insufficient isolation between critical systems
- Complexity in disaster recovery procedures
- Reliance on affected infrastructure for management and orchestration

7. ENGINEERING LESSONS

- ✓ Design control plane for multi-region resilience
- ✓ Remove single data center dependencies
- ✓ Test disaster recovery regularly and realistically
- ✓ Ensure out-of-band access to critical systems
- ✓ Map and review hidden dependencies
- ✓ Have clear communication and customer updates
- ✓ Design for power and infrastructure failures

2. SIMPLIFIED ARCHITECTURE



Loss of power impacted control-plane services, leading to service disruptions across multiple products and regions.

4. TIMELINE (UTC)

- | | | |
|-------|--|--|
| 14:48 | | Power failure occurred at data center |
| 14:52 | | Internal systems started to degrade |
| 15:01 | | Multiple Cloudflare services reported issues |
| 15:27 | | Engineers identified power-related impact |
| 16:20 | | Failover and recovery processes initiated |
| 17:49 | | Services gradually restored |
| 18:42 | | Majority of services back to normal |

* Times are approximate based on publicly available reports.

6. WHAT MADE IT WORSE

- Control-plane services were dependent on the affected data center
- Failover processes took longer than expected
- Some recovery tools were not accessible
- Complex dependencies increased recovery time
- Simultaneous impact on multiple services

8. KEY TAKEAWAYS

- ✓ Infrastructure failures can still cause global impact
- ✓ Control-plane dependencies can be a single point of failure
- ✓ Disaster recovery must be tested end to end
- ✓ Out-of-band access is critical during major outages
- ✓ Understand and document all dependencies
- ✓ Build for resilience, not just high availability
- ✓ Expect the unexpected

THE CASCADING OVERLOAD INCIDENT

CAPACITY EXHAUSTION, RETRIES, POSITIVE FEEDBACK AND CASCADING FAILURE

1. INCIDENT OVERVIEW

A major online service experienced a large-scale outage due to a sudden surge in traffic that exceeded system capacity. Automatic retries, timeouts and cascading requests created a positive feedback loop, causing multiple services to become overloaded. This led to widespread service degradation and outages across the platform and dependent services.

3. IMPACT

-  Many users experienced slow performance or complete service outages
-  Affected multiple products and dependent services
-  Lasted for several hours
-  High error rates across the platform
-  Impact extended to third-party services that depended on the platform

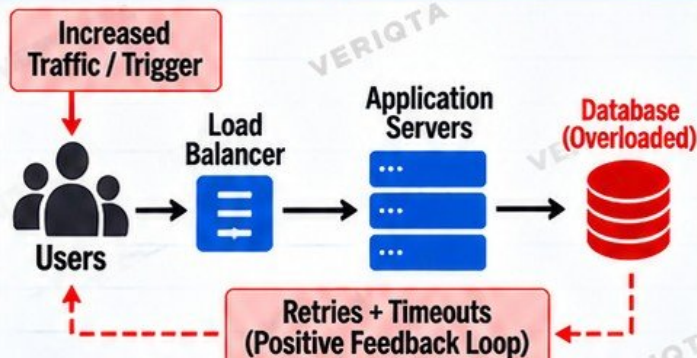
5. ROOT CAUSES

- Sudden traffic spike exceeded system capacity
- Automatic retries and short timeouts increased load
- Lack of proper rate limiting and backpressure
- Cascading failures across dependent services
- Insufficient capacity buffers
- Inadequate protection against positive feedback loops

7. ENGINEERING LESSONS

- ✓ Implement rate limiting and backpressure
- ✓ Use circuit breakers and bulkheads
- ✓ Set reasonable timeouts and retry policies
- ✓ Design for graceful degradation
- ✓ Test for traffic spikes and failure scenarios
- ✓ Monitor key metrics (latency, error rate, saturation)
- ✓ Prepare runbooks for overload scenarios

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

- | | |
|--|---|
| <p>12:00</p> <p>12:05</p> <p>12:10</p> <p>12:16</p> <p>12:25</p> <p>12:40</p> <p>14:10</p> <p>16:30</p> <p>18:00</p> | <p> Traffic started to increase significantly</p> <p> Latency increased, error rates began to rise</p> <p> Automatic retries amplified the load</p> <p> Key services became overloaded</p> <p> Cascading failures across dependent services</p> <p> Majority of services were unavailable</p> <p> Mitigation steps reduced the load</p> <p> Services gradually recovered</p> <p> Full recovery completed</p> |
|--|---|

* Times are approximate based on publicly available reports.

6. WHAT MADE IT WORSE

- Aggressive retry policies
- Synchronous dependencies between services
- Lack of circuit breakers
- Slow failure detection and traffic shedding
- Insufficient load testing for traffic spikes
- Cascading failures across the entire system

8. KEY TAKEAWAYS

- ✓ Retries can turn a small problem into a major outage
- ✓ Systems must be designed to handle traffic spikes
- ✓ Use circuit breakers to prevent cascading failures
- ✓ Monitor and alert on early signs of overload
- ✓ Design services to fail fast and recover quickly
- ✓ Test your systems under real-world conditions
- ✓ Assume failure and build for resilience

**CONTROL RETRIES.
PROTECT YOUR SYSTEMS.
BUILD FOR RESILIENCE.**

DATABASE CONNECTION POOL EXHAUSTION

CONNECTION SATURATION TURNS APPLICATION SLOWDOWN INTO DATABASE FAILURE

1. INCIDENT OVERVIEW

An online application experienced increasing latency due to a surge in traffic and inefficient connection handling. The application exhausted its database connection pool, causing all available connections to be used. New requests could not get a database connection, leading to application errors, cascading retries and eventually database instability and failure.

3. IMPACT



Users experienced slow performance and errors



Some services became completely unavailable



Lasted for several hours (varies by incident)



Increased load led to database instability and, in some cases, a full outage



Affected revenue, customer trust and business operations

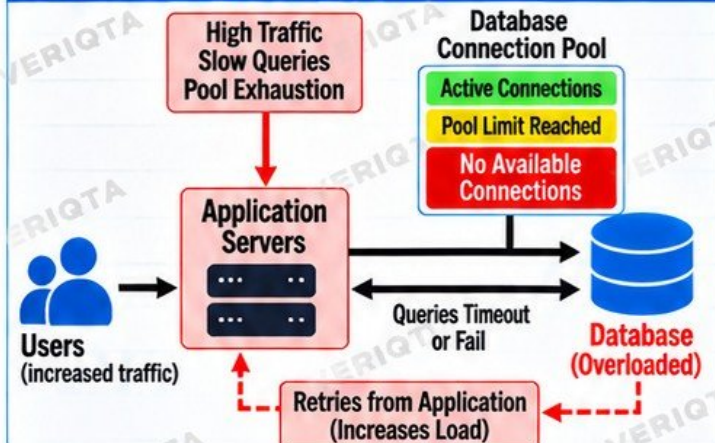
5. ROOT CAUSES

- Traffic surge or sudden load increase
- Connection pool size too small
- Long-running or slow database queries
- Connections not released properly (connection leaks)
- Excessive retries and lack of backoff
- Insufficient monitoring and alerting
- Inadequate load testing for peak traffic

7. ENGINEERING LESSONS

- ✓ Right-size and tune connection pools
- ✓ Implement proper timeout and retry with exponential backoff
- ✓ Fix slow queries and add database indexes
- ✓ Ensure connections are always released (use connection pooling best practices)
- ✓ Use circuit breakers and bulkheads
- ✓ Monitor key metrics (active connections, wait time, query time)
- ✓ Test for realistic traffic and failure scenarios
- ✓ Have a clear runbook for database connection issues

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

10:00		Traffic started to increase
10:15		Application latency began to rise
10:20		Database connection pool reached maximum
10:22		New requests started to fail
10:25		Automatic retries increased the load
10:30		Database became unstable (high CPU / memory)
10:40		Majority of application requests failing
11:15		Engineers identified connection pool exhaustion
12:10		Mitigation implemented (scaled, tuned, reduced retries)
13:00		Services gradually recovered

* Times are approximate and may vary by incident.

6. WHAT MADE IT WORSE

- Aggressive retry logic from the application
- Multiple services sharing the same database
- Lack of connection timeouts
- Inefficient queries and missing indexes
- No circuit breakers or rate limiting
- Slow detection and response
- Database resource limits (CPU, memory, connections)

8. KEY TAKEAWAYS

- ✓ Connection pool exhaustion can turn a slowdown into a full outage
- ✓ Retries can make the problem much worse
- ✓ Proper connection management is critical
- ✓ Monitoring and alerting should cover database connection health
- ✓ Design for failure with circuit breakers and backoff
- ✓ Load test with realistic traffic patterns
- ✓ Small configuration changes can have a big impact
- ✓ Be prepared to scale, tune and optimize quickly

STABLE SYSTEMS
HAPPY USERS
STRONGER ENGINEERS

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



RETRY STORM

SMALL RETRIES
BIG OUTAGES
TIMEOUTS TRIGGER RETRIES, RETRIES CREATE MORE LOAD, RECOVERY BECOMES IMPOSSIBLE

1. INCIDENT OVERVIEW

A widely used online service experienced a major outage after an initial increase in latency caused clients to timeout and retry requests. These retries generated a massive surge in traffic, overwhelming the already struggling services. The retry storm created a positive feedback loop, making the situation worse and preventing recovery until retries were stopped and traffic reduced.

3. IMPACT

-  Service became slow or completely unavailable
-  Affected millions of users (depending on the service)
-  Lasted for hours
-  High error rates and increased latency across the platform
-  Impacted revenue, customer trust and business operations

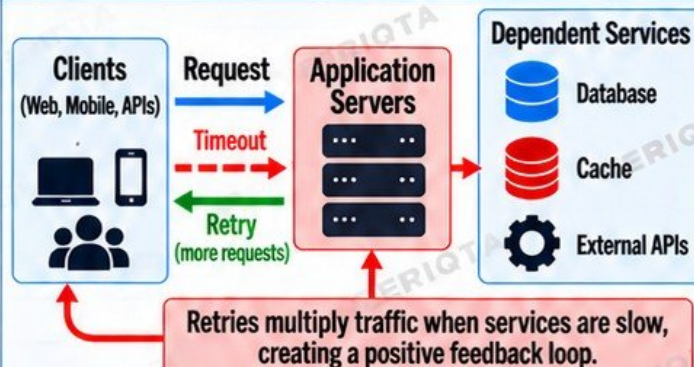
5. ROOT CAUSES

- Aggressive retry policies (immediate retries)
- Lack of exponential backoff
- No jitter (randomization) in retries
- Short client timeouts
- Insufficient rate limiting
- Cascading failures across dependent services
- Lack of circuit breakers
- Inadequate load testing for retry scenarios

7. ENGINEERING LESSONS

- ✓ Use exponential backoff with jitter for retries
- ✓ Set reasonable timeouts
- ✓ Implement circuit breakers
- ✓ Apply rate limiting and bulkheads
- ✓ Do not retry on non-retryable errors (e.g. 4xx)
- ✓ Load test retry scenarios
- ✓ Monitor retry rates and traffic patterns
- ✓ Have a clear plan to quickly disable or reduce retries

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

- | | |
|-------|---|
| 10:00 | Latency started to increase |
| 10:02 | Clients began to timeout |
| 10:03 | Automatic retries significantly increased traffic |
| 10:05 | Services became overloaded |
| 10:08 | Error rates spiked across multiple services |
| 10:15 | Majority of requests were failing |
| 10:40 | Engineers identified retry storm as the cause |
| 11:20 | Retries were disabled/limited |
| 12:10 | Traffic gradually reduced |
| 13:00 | Services recovered |

* Times are approximate and may vary by incident.

6. WHAT MADE IT WORSE

- Retries from multiple clients at the same time
- Synchronous dependencies between services
- No backoff or jitter in retry logic
- Clients retrying on non-retryable errors
- Insufficient protection (rate limiting, circuit breakers)
- Services sharing the same infrastructure
- Monitoring and alerts triggered too late
- The retry storm continued even during recovery

8. KEY TAKEAWAYS

- ✓ Retries can turn a small issue into a major outage
- ✓ Timeouts and retries must be carefully designed
- ✓ Use exponential backoff and jitter
- ✓ Protect your services with circuit breakers and rate limits
- ✓ Monitor and alert on retry behavior
- ✓ Test for failure scenarios, not just normal traffic
- ✓ Be ready to quickly throttle or disable retries
- ✓ Resilience requires planning, not luck

THINK AHEAD.
DESIGN FOR FAILURE.
BUILD FOR RESILIENCE.



kubernetes

KUBERNETES CRASHLOOP CASCADE

BAD ROLLOUT, FAILING WORKLOADS, PROBES, RESTART BEHAVIOR AND CAPACITY LOSS

1. INCIDENT OVERVIEW

A production Kubernetes environment experienced a major outage after a bad rollout introduced a defect that caused pods to fail startup checks. Liveness probes triggered continuous restarts, creating CrashLoopBackOff conditions across many pods. The repeated restarts consumed cluster resources, leading to node pressure, reduced capacity and service outages across multiple applications.

3. IMPACT



Multiple services became unavailable or degraded



Affected internal and external users



Lasted for several hours



High resource usage across the cluster (CPU, memory, disk)



Automatic restarts made recovery slower



Impact on revenue, customer trust and business operations

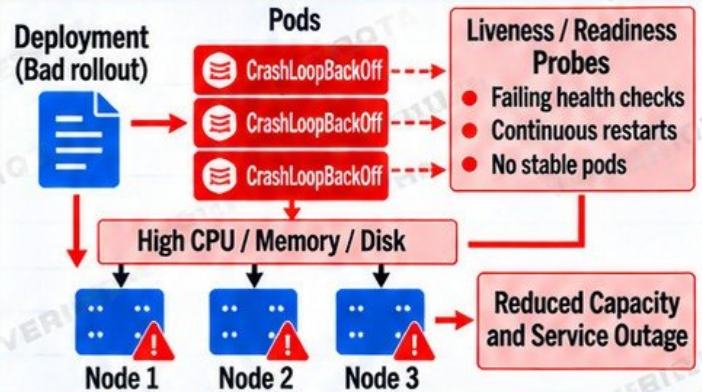
5. ROOT CAUSES

- Bad deployment (defective image or configuration)
- Failing liveness or readiness probes
- Aggressive restart behavior
- Insufficient resource requests and limits
- Lack of rollout safety checks (canary or staged rollout)
- Cascading resource exhaustion on cluster nodes
- Insufficient monitoring and alerting
- Human error in release process

7. ENGINEERING LESSONS

- ✓ Use startup, liveness and readiness probes correctly
- ✓ Implement canary or staged rollouts
- ✓ Set proper resource requests and limits
- ✓ Test deployments in staging environments
- ✓ Monitor pod restarts, node resources and probe failures
- ✓ Use deployment safeguards (maxUnavailable, maxSurge)
- ✓ Have a fast rollback procedure
- ✓ Design for failure and assume things will break

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

09:00		New version deployed (bad rollout)
09:02		Pods started failing health checks
09:05		CrashLoopBackOff events increased
09:10		Cluster resource usage began to spike
09:18		Multiple services became unavailable
09:30		Engineers identified the issue
10:05		Rollback initiated
10:25		Pods stabilized
10:40		Services gradually recovered
11:20		Full recovery completed

* Times are approximate and may vary by incident.

6. WHAT MADE IT WORSE

- Liveness probes set too aggressive
- No startup probe for slow applications
- All replicas failing at the same time
- Cluster autoscaler could not react quickly enough
- Other workloads were affected due to resource contention
- Lack of proper rollout strategy (no canary/blue-green)
- Insufficient pre-deployment testing
- Delay in detecting and stopping the rollout

8. KEY TAKEAWAYS

- ✓ A bad rollout can take down your entire platform
- ✓ Probe configuration matters
- ✓ CrashLoopBackOff can consume significant resources
- ✓ Cluster capacity can be lost quickly
- ✓ Monitoring and alerting must catch issues early
- ✓ Use progressive delivery for safer deployments
- ✓ Test, validate and have a rollback plan
- ✓ Build resilience into your Kubernetes operations

STABLE CLUSTERS.
RELIABLE SERVICES.
STRONGER ENGINEERS.

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



kubernetes

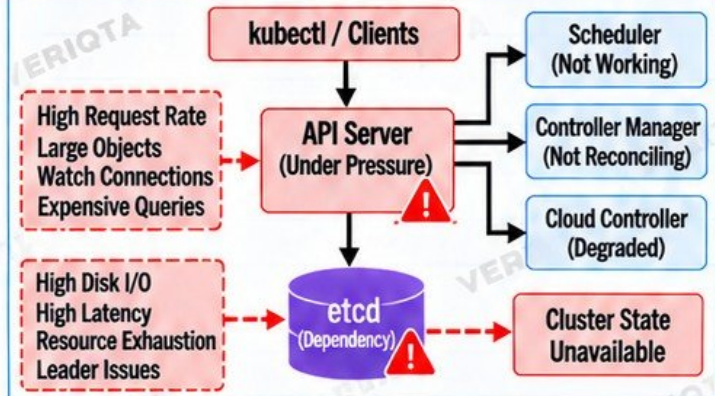
KUBERNETES CONTROL PLANE INCIDENT

API SERVER PRESSURE, ETCD DEPENDENCY AND LOSS OF CLUSTER CONTROL

1. INCIDENT OVERVIEW

A Kubernetes cluster experienced a major outage after the control plane became unhealthy due to high API server load and etcd performance issues. The API server became unresponsive, controllers could not reconcile, new pods could not be scheduled and cluster management operations failed. In some cases, etcd instability or unavailability led to a complete loss of cluster control, making the cluster effectively unusable until etcd was restored and control plane components recovered.

2. SIMPLIFIED ARCHITECTURE



3. IMPACT



Cluster management operations failed (kubectl not working)



New deployments could not be created



Existing workloads were affected over time



Controllers could not reconcile desired state



In some cases, the entire cluster became unavailable



Impact on development, CI/CD, internal platforms and customer-facing services

4. TIMELINE (UTC)

09:00	Increased API server latency
09:05	etcd latency started to rise
09:10	kubectl commands became slow
09:15	Controllers began to fall behind
09:20	New pod creations started to fail
09:25	API server became unresponsive
09:30	Cluster effectively lost
10:10	Investigation identified etcd issues
11:00	etcd restored and control plane recovered
12:00	Cluster operations back to normal

* Times are approximate and may vary by incident.

5. ROOT CAUSES

- High request rate or expensive API queries
- Large number of watch connections
- etcd performance degradation (CPU, disk, network)
- etcd leader failure or quorum loss
- Insufficient control plane resources (CPU, memory, disk)
- Misconfigured components or upgrades
- Inefficient controllers or operators
- Lack of rate limiting and request prioritization

6. WHAT MADE IT WORSE

- No request throttling or API rate limiting
- Large objects stored in etcd
- Too many controllers and operators
- Under-provisioned etcd and control plane nodes
- Cluster upgrades during high load
- Lack of monitoring and alerting on etcd and API server
- Slow detection and response
- No backup or disaster recovery plan for etcd

7. ENGINEERING LESSONS

- ✓ Monitor API server and etcd metrics (latency, QPS, disk, DB size)
- ✓ Set request rate limits and use priority and fairness
- ✓ Keep objects small and avoid unnecessary writes to etcd
- ✓ Right-size control plane and etcd resources
- ✓ Test upgrades in non-production environments
- ✓ Implement etcd backups and disaster recovery procedures
- ✓ Monitor controller health and queue lengths
- ✓ Have a runbook for control plane and etcd recovery

8. KEY TAKEAWAYS

- ✓ The control plane is critical for the entire cluster
- ✓ etcd health directly affects cluster stability
- ✓ API server can become a bottleneck under load
- ✓ Prevention requires monitoring, limits and right-sizing
- ✓ Detect issues early and have a recovery plan
- ✓ Backups and DR for etcd are essential
- ✓ Design for failure, not just normal operations
- ✓ A healthy control plane keeps your platform and business running

OBSERVE. PREPARE.
BUILD MORE RESILIENT SYSTEMS.

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta

veriqta

veriqta

veriqta

veriqta

veriqta

DNS FAILURE THAT LOOKS LIKE APPLICATION FAILURE

RESOLUTION FAILURE, CACHES, TTLS, AUTHORITATIVE DNS AND MISLEADING SYMPTOMS

1. INCIDENT OVERVIEW

A widely used online service experienced a major outage caused by a DNS issue. Users saw application errors, but the real problem was DNS resolution failure. The domain could not be resolved correctly due to misconfiguration and propagation issues at the authoritative DNS provider. Caches, TTL values and resolver behavior caused inconsistent results, making the outage appear like an application failure instead of a DNS failure.

3. IMPACT



Users could not access the service



Seen as application downtime



Lasted for several hours
(varied by region due to caching)



Affected multiple products and regions



Caused confusion, unnecessary application rollbacks and increased support tickets



Impact on revenue, customer trust and business operations

5. ROOT CAUSES

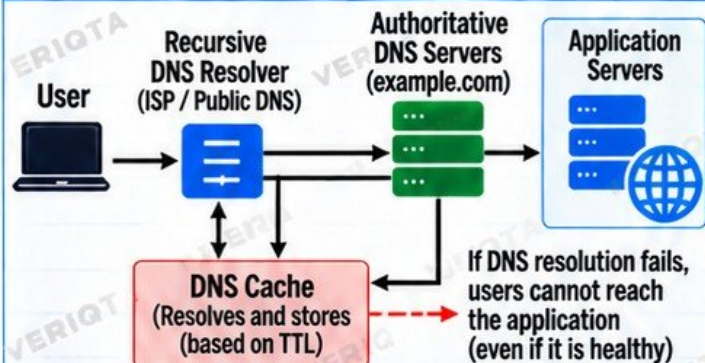
- Misconfigured DNS records (e.g. wrong IP, deleted record)
- Authoritative DNS server issues
- Human error during DNS changes
- Low TTL values or incorrect TTL strategy
- Negative caching (NXDOMAIN)
- Propagation delays across global DNS resolvers
- Lack of change validation and testing
- Insufficient monitoring and alerting for DNS

7. ENGINEERING LESSONS

- ✓ Validate DNS changes before deployment
- ✓ Use appropriate TTL values based on risk
- ✓ Monitor DNS resolution from multiple regions
- ✓ Set up alerts for DNS failures and NXDOMAIN errors
- ✓ Have a rollback plan for DNS changes
- ✓ Understand cache behavior and propagation time
- ✓ Test from different networks and DNS resolvers
- ✓ Document DNS architecture and ownership
- ✓ Include DNS in incident response runbooks

**DNS KEEPS THE INTERNET
RUNNING. MONITOR IT.
RESPECT IT. BE PREPARED.**

2. SIMPLIFIED ARCHITECTURE



4. TIMELINE (UTC)

- 08:00 | DNS configuration change deployed
- 08:05 | Authoritative DNS started returning errors or empty responses
- 08:10 | Users began reporting application errors
- 08:20 | DNS resolution failures increased globally
- 08:45 | Engineers initially investigated the application
- 09:15 | Identified DNS as the root cause
- 10:30 | DNS configuration corrected
- 11:00 | DNS propagation in progress (due to TTL and caches)
- 13:00 | Service fully restored for most users

* Times are approximate and may vary by incident.

6. WHAT MADE IT WORSE

- Long TTL values causing slow recovery
- Cached negative responses (NXDOMAIN)
- Inconsistent results across regions and resolvers
- Misleading application-level error messages
- Engineers focused on the application instead of DNS
- Lack of real-time DNS monitoring and alerts
- Dependence on a single DNS provider
- No clear DNS rollback plan
- Third-party services also affected (e.g. email, APIs, CDN)

8. KEY TAKEAWAYS

- ✓ DNS can take down your service even if your application is healthy
- ✓ Always consider DNS when users report application errors
- ✓ TTL and caching behavior directly affect recovery time
- ✓ Monitor DNS from multiple regions and providers
- ✓ Validate and test DNS changes before going live
- ✓ Have a clear rollback and communication plan
- ✓ Don't rely on a single DNS provider
- ✓ Educate your team on DNS basics and troubleshooting
- ✓ Be prepared for inconsistent behavior during DNS incidents



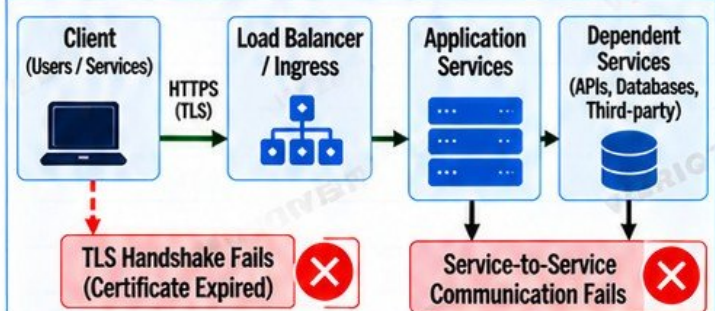
CERTIFICATE EXPIRATION PRODUCTION OUTAGE

TLS DEPENDENCY, FAILED RENEWAL, MONITORING GAPS AND SERVICE-TO-SERVICE FAILURE

1. INCIDENT OVERVIEW

A production outage occurred when a critical TLS certificate expired and was not renewed. This caused clients and internal services to fail TLS handshakes, resulting in service-to-service failures and user-facing errors. The issue was not detected early due to inadequate monitoring and alerting for certificate expiration. Multiple dependent services were affected, leading to a widespread outage until a new certificate was issued and deployed.

2. SIMPLIFIED ARCHITECTURE



3. IMPACT

-  Users could not access the service
-  Internal and external services failed
-  Lasted for several hours (depending on detection and response time)
-  Affected multiple products and regions
-  Service-to-service dependencies also failed
-  Impact on revenue, customer trust and business operations

4. TIMELINE (UTC)

- 08:00 | Certificate expired (not renewed)
- 08:01 | TLS handshake failures began
- 08:05 | Users started reporting errors
- 08:10 | Internal services also failed to connect
- 08:20 | On-call engineer alerted
- 08:35 | Investigation identified expired certificate
- 09:15 | New certificate issued
- 09:40 | Certificate deployed across all services
- 10:10 | Services gradually recovered
- 11:00 | Full recovery completed

* Times are approximate and may vary by incident.

5. ROOT CAUSES

- Expired TLS certificate (failed renewal)
- Lack of monitoring and alerting for certificate expiration
- No automated renewal process (or renewal failed)
- Poor visibility into certificate dependencies
- Human error or missed notifications
- Short certificate validity period without proper tracking
- Multiple services using the same expired certificate
- Inadequate testing of certificate renewal process

6. WHAT MADE IT WORSE

- No alerts before expiration
- Multiple services depended on the same certificate
- Lack of certificate inventory and ownership
- Manual renewal process
- Third-party services also affected
- No fallback or redundant certificates
- Propagation delays after renewal
- Incomplete rollout across all environments
- Inconsistent monitoring between teams
- High traffic during the incident increased error rates

7. ENGINEERING LESSONS

- ✓ Implement automated certificate renewal (e.g. Let's Encrypt, ACM, cert-manager)
- ✓ Set up monitoring and alerts for certificate expiration
- ✓ Maintain a certificate inventory (owner, expiry date, usage)
- ✓ Use longer validity periods where appropriate
- ✓ Test certificate renewal process in staging environments
- ✓ Have a clear runbook for certificate expiry and renewal
- ✓ Use separate certificates for different services where possible
- ✓ Monitor service-to-service TLS dependencies
- ✓ Regularly audit and review certificate configurations
- ✓ Don't rely on manual processes for critical certificates

8. KEY TAKEAWAYS

- ✓ Certificates are critical infrastructure
- ✓ Expiration can cause complete outages
- ✓ Monitor, automate and have alerts
- ✓ Service-to-service TLS failures can be as impactful as user-facing failures
- ✓ Maintain visibility and ownership of all certificates
- ✓ Test renewal processes regularly
- ✓ Have a clear rollback and recovery plan
- ✓ Use tools like cert-manager, AWS ACM, or automated renewal systems
- ✓ Small operational gaps can lead to major outages
- ✓ Security and reliability go hand in hand

**AUTOMATE RENEWALS.
MONITOR EXPIRY.
PREVENT OUTAGES.**

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**CERTIFICATES EXPIRE.
DOWNTIME SHOULDN'T.**



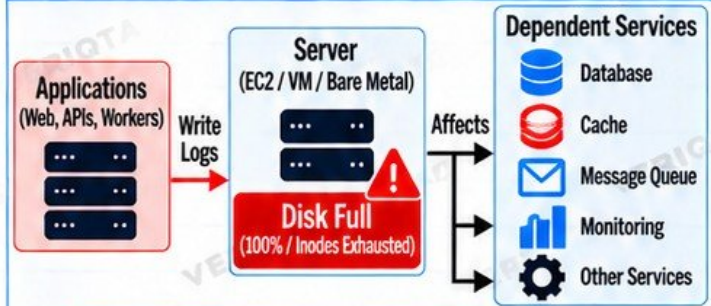
DISK FULL AT 02:00

**NO SPACE
NO SERVICE**
LOGS, INODE/STORAGE EXHAUSTION, DATABASE IMPACT AND SECONDARY FAILURES

1. INCIDENT OVERVIEW

At 02:00, a critical server ran out of disk space due to uncontrolled log growth and exhausted storage inodes. The disk full condition caused applications to fail, database writes to stop and several dependent services to become unavailable. Monitoring did not alert early enough, and log rotation was misconfigured. The issue quickly escalated, leading to a major production outage with secondary failures across multiple systems.

2. SIMPLIFIED ARCHITECTURE



Disk full on one server can cause failures across multiple services.

3. IMPACT



Services became slow or completely unavailable



Users experienced errors (5xx, timeouts)



Lasted for several hours



Affected multiple products and regions



Database writes failed, causing data and application errors



Impact on revenue, customer trust and business operations

4. TIMELINE (UTC)

01:50	Disk usage reached 90%
02:00	Disk became 100% full (applications started failing)
02:05	Database writes failed
02:10	Multiple services reported errors
02:20	On-call team alerted
02:30	Initial investigation began
02:50	Identified log growth and inode exhaustion
03:10	Freed up disk space and fixed log rotation
03:30	Services gradually recovered
04:00	All systems stable

* Times are approximate and may vary by incident.

5. ROOT CAUSES

- Uncontrolled log growth
- No or misconfigured log rotation
- Disk space monitoring gaps
- Inode exhaustion (many small files)
- Large temporary files not cleaned up
- Database logs and backups consuming space
- Lack of automated cleanup policies
- No alerts or alerts set too high
- Poor capacity planning
- Human error or recent configuration change

6. WHAT MADE IT WORSE

- Monitoring did not alert early
- Logs filled up faster than expected
- Inode exhaustion prevented new files from being created
- Applications could not write logs and started failing
- Database writes failed, causing cascading errors
- Other services (cache, queue, monitoring) were affected
- Disk cleanup took time due to large files
- No automated remediation process
- Lack of runbooks for disk full scenarios
- Occurred during low-staff hours (02:00)

7. ENGINEERING LESSONS

- ✓ Implement proper log rotation and retention
- ✓ Monitor disk usage and inode usage (set meaningful alerts)
- ✓ Set up automated cleanup for temporary files
- ✓ Use separate partitions for logs, data and system files
- ✓ Implement capacity planning and growth forecasts
- ✓ Have alerts and dashboards for disk and inode metrics
- ✓ Test disk full scenarios in non-production
- ✓ Document and practice the runbook
- ✓ Keep logs at appropriate levels (avoid excessive logging)
- ✓ Review recent changes during incident investigation

8. KEY TAKEAWAYS

- ✓ Disk full can take down critical services quickly
- ✓ Monitor both disk space and inodes
- ✓ Log rotation and retention are essential
- ✓ Databases are highly sensitive to disk space issues
- ✓ One server issue can cause widespread impact
- ✓ Automate cleanup and alerting
- ✓ Have a clear runbook for disk full incidents
- ✓ Test and validate monitoring
- ✓ Plan for capacity before it becomes a crisis
- ✓ Small operational gaps can cause major outages

**MONITOR. MAINTAIN.
PREVENT OUTAGES.
DISK SPACE MATTERS.**



MEMORY LEAK AND OOMKILL CASCADE

GRADUAL DEGRADATION, MEMORY PRESSURE, KERNEL/CONTAINER TERMINATION AND RESTART LOOPS

1. INCIDENT OVERVIEW

A production service developed a memory leak, causing gradual increase in memory usage over time. As memory consumption grew, the node and containers experienced memory pressure. Eventually, the kernel (or Kubernetes) terminated the process using the OOM killer. The container restarted, but the underlying issue remained, leading to repeated OOM kills, restart loops and service instability. The issue affected multiple replicas and downstream services, causing a wider production impact.

3. IMPACT



Service became slow or unavailable



Users experienced errors (5xx, timeouts)



Lasted for several hours



Affected multiple services and regions



Increased infrastructure costs due to restarts



Impact on revenue, customer trust and business operations

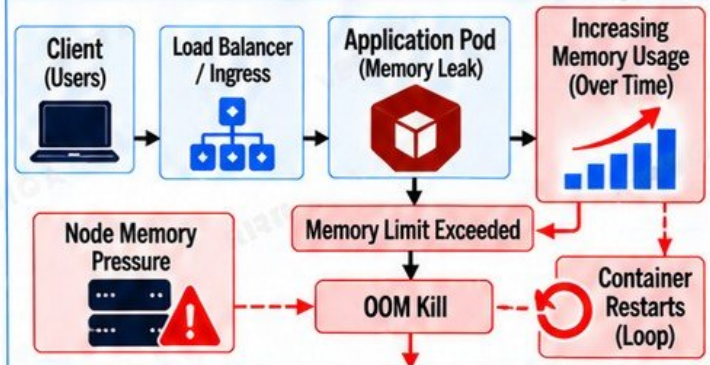
5. ROOT CAUSES

- Memory leak in application code
- Insufficient memory limits and requests
- Lack of memory usage monitoring and alerting
- No automated detection of abnormal memory growth
- Inadequate load testing and long-running tests
- Configuration or dependency causing memory growth (e.g. cache, connection pool, unclosed resources)
- Slow response to early warning signs
- Multiple replicas affected due to same issue

7. ENGINEERING LESSONS

- ✓ Monitor memory usage and set meaningful alerts
- ✓ Set appropriate memory requests and limits
- ✓ Use profiling and heap dumps to detect memory leaks
- ✓ Run long-duration and load tests
- ✓ Implement automated detection for abnormal memory growth
- ✓ Use canary deployments and gradual rollouts
- ✓ Have a runbook for OOM kills and memory issues
- ✓ Optimize application code and dependencies
- ✓ Observe container and node memory metrics (cAdvisor, Prometheus)
- ✓ Don't rely only on restarts to fix problems

2. SIMPLIFIED ARCHITECTURE



The container keeps restarting, but the memory leak remains, causing a continuous failure cycle.

4. TIMELINE (UTC)

22:00	Memory usage started increasing gradually
23:00	Service became slower
00:30	High memory usage alerts triggered (not acted on)
01:30	Node memory pressure detected
02:10	First OOM kill
02:12	Container restarted
02:20	Repeated OOM kills and restart loops began
02:40	Multiple replicas affected
03:30	Downstream services started failing
04:15	Root cause identified (memory leak in application)
05:00	Fix deployed and traffic stabilized

* Times are approximate and may vary by incident.

6. WHAT MADE IT WORSE

- No alerts or alerts ignored
- Auto-restart masked the real problem
- Insufficient logging and observability
- Multiple services running on the same node
- High traffic increased memory consumption faster
- Lack of memory limits (or limits set too high)
- Dependency services also affected
- No canary or staged rollout
- Restart loops consumed additional resources
- Delayed investigation and mitigation

8. KEY TAKEAWAYS

- ✓ Memory leaks can be silent killers in production
- ✓ Monitor, alert and act early on memory growth
- ✓ Set and enforce memory limits for all containers
- ✓ Auto-restarts are not a fix, they can hide the problem
- ✓ Use observability tools to get visibility (metrics, logs, traces)
- ✓ Understand the OOM killer and restart behavior
- ✓ Test for memory issues before going live
- ✓ A single faulty service can impact many others
- ✓ Have a clear incident response and rollback plan
- ✓ Build a culture of proactive monitoring and resilience

OBSERVE. DIAGNOSE.
FIX. PREVENT.
STRONGER SYSTEMS.

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

SMALL MEMORY LEAKS
CAN CREATE
BIG OUTAGES.



DEPLOYMENT THAT PASSED CI BUT **BROKE PRODUCTION**

CANARY GAPS, CONFIGURATION DIFFERENCES, ROLLBACK AND CHANGE SAFETY

1. INCIDENT OVERVIEW

A new version of the application passed all CI checks, including build, tests, security scans and deployments to non-production environments. After deployment to production, errors increased, key features failed and users experienced outages. The issue was caused by configuration differences, missing canary validation and insufficient production-like testing. The rollback took longer than expected, leading to a significant business impact.

3. IMPACT



Users experienced errors and service outages



Key features were unavailable



Lasted for several hours



Affected multiple services and regions



Loss of customer trust and revenue

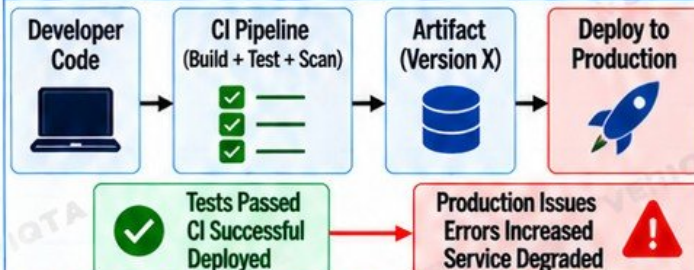


Increased operational workload



Emergency rollback required

2. SIMPLIFIED ARCHITECTURE



Missing or Weak Controls

No real canary validation

Configuration differences

Insufficient production-like tests

4. TIMELINE (UTC)

10:00	Code merged to main branch
10:05	CI pipeline ran and passed
10:20	Artifact built and deployed to production
10:28	First errors reported by monitoring
10:35	User complaints started
10:45	Investigation began
11:10	Root cause identified (configuration difference)
11:30	Rollback initiated
11:50	Services recovered
12:15	Full recovery completed

* Times are approximate and may vary by incident.

5. ROOT CAUSES

- Configuration differences between environments
- No or insufficient canary deployment
- Tests did not cover real production scenarios
- Missing feature flags or kill switches
- Inadequate validation of infrastructure dependencies
- Manual changes in production not replicated in CI
- Insufficient monitoring and alerting for new version
- Lack of rollback automation and practice
- Pressure to deploy quickly

6. WHAT MADE IT WORSE

- CI tests did not fully match production environment
- No canary or gradual rollout
- Missing or incorrect feature flags
- Lack of automated rollback
- Manual configuration changes in production
- Insufficient observability for the new version
- Delayed detection and alerting
- Multiple dependent services affected
- High traffic at the time of deployment
- Lack of a tested rollback plan

7. ENGINEERING LESSONS

- ✓ Make production environments as close to real as possible
- ✓ Use canary deployments and gradual rollouts
- ✓ Implement feature flags and kill switches
- ✓ Automate rollback and test rollback regularly
- ✓ Validate configuration and environment differences
- ✓ Include integration and end-to-end tests
- ✓ Monitor new deployments closely with clear alerts
- ✓ Document and follow a change management process
- ✓ Avoid manual changes in production
- ✓ Prioritize safety over speed

8. KEY TAKEAWAYS

- ✓ Passing CI does not guarantee production success
- ✓ Environment differences can break your application
- ✓ Canary deployments reduce risk
- ✓ Feature flags and automated rollbacks are essential
- ✓ Test in production-like environments
- ✓ Monitor closely after every deployment
- ✓ Have a clear and tested rollback plan
- ✓ Treat every deployment as a potential failure
- ✓ Invest in observability and alerting
- ✓ A culture of careful, incremental change keeps systems stable

SAFE DEPLOYMENTS
BUILD TRUST.

VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

DEPLOY SMART.
ROLLBACK FASTER.
BUILD LONGER.


**INCIDENT
COMMANDER**

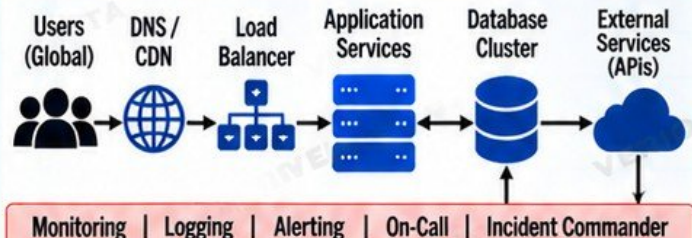
THE INCIDENT COMMANDER CASE

FULL MULTI-SYSTEM INCIDENT FROM DETECTION THROUGH MITIGATION, RECOVERY AND POSTMORTEM

1. INCIDENT OVERVIEW

A real-world, multi-system incident where a production outage affected users across multiple regions. This case follows the complete incident lifecycle from initial detection, through investigation, mitigation, recovery and postmortem. It demonstrates the role of the Incident Commander in coordinating people, systems and communication during a high-severity incident.

2. INCIDENT ENVIRONMENT (SIMPLIFIED)



3. IMPACT

- Users could not access key services
- Service downtime across multiple regions
- Lasted for several hours
- Affected multiple products and customers
- Revenue loss and degraded customer trust
- Multiple dependent systems impacted
- Required cross-functional response (Engineering, Product, Support, Comms)

4. INCIDENT TIMELINE (UTC)

10:02		Alert triggered (increased error rate)
10:05		Incident Commander (IC) assigned
10:08		Initial investigation and impact assessment
10:15		Confirmed multi-service impact
10:25		War room activated (engineering, product, support)
10:40		Root cause identified
11:05		Mitigation implemented
11:35		Services recovering
12:10		All services restored
12:30		Monitoring and validation complete
13:00		Customer communication updated
24:00		Postmortem started

* Times are approximate and may vary by incident.

5. ROOT CAUSE (EXAMPLE)

- Configuration change triggered a cascade failure
- A dependency service became unavailable
- Insufficient failure handling and timeouts
- Monitoring did not detect the real issue early enough
- Lack of automated rollback
- Inadequate testing in a production-like environment
- Multiple systems amplified the impact
- Process and communication gaps

6. WHAT MADE IT WORSE

- Slow initial detection and alerting
- Insufficient observability across services
- Missing circuit breakers and graceful degradation
- Manual intervention required in multiple systems
- High traffic due to peak usage
- Third-party service also experienced issues
- Unclear ownership of critical components
- Delayed customer communication
- Lack of recent incident drills

7. ENGINEERING LESSONS

- ✓ Invest in end-to-end observability (metrics, logs, traces)
- ✓ Set up meaningful alerts and on-call runbooks
- ✓ Use circuit breakers, retries and timeouts
- ✓ Automate rollback and recovery where possible
- ✓ Test failure scenarios in staging and production-like environments
- ✓ Document dependencies and ownership
- ✓ Run regular incident response drills
- ✓ Improve cross-team communication
- ✓ Update monitoring based on lessons learned
- ✓ Treat every incident as a learning opportunity

8. KEY TAKEAWAYS

- ✓ The Incident Commander provides clear leadership and coordination
- ✓ Focus on impact, not blame
- ✓ Good observability and alerting reduce detection time
- ✓ Automation and better failure handling limit blast radius
- ✓ Communication is critical (internal and external)
- ✓ Runbooks and prepared teams respond faster
- ✓ Postmortems drive long-term improvement
- ✓ Resilience is built before an incident, not during
- ✓ A multi-system incident tests people, process and technology
- ✓ Learn, improve, and be ready for the next one

**PREPARE. RESPOND.
RECOVER. IMPROVE.
THAT'S ENGINEERING.**
VERIQTA

KNOWLEDGE FOR A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**INCIDENTS HAPPEN.
BETTER ENGINEERS
LEARN FROM THEM.**